

Continuando o StreamTube com IA — Fase 03: Upload e Processamento de Vídeos

Descrição

O **StreamTube** é a plataforma de compartilhamento de vídeos que você acompanhou sendo construída ao longo do curso. O professor entregou as **Fases 01 (configuração base)** e **02 (autenticação, usuários e canais)**, backend e frontend, seguindo o workflow de desenvolvimento orientado por IA do curso.

Seu desafio é **dar continuidade ao projeto implementando a próxima fase da sequência, a Fase 03 — Upload e Processamento de Vídeos**, por completo. Diferente de uma feature simples de CRUD, a Fase 03 é um desafio de engenharia por si só: envolve **armazenamento de arquivos grandes, processamento assíncrono em fila, um worker de vídeo, streaming e infraestrutura nova em Docker**. Você vai conduzir tudo isso usando IA como ferramenta principal, seguindo o workflow do projeto de ponta a ponta.

Este é um desafio de **backend**: a entrega é a API, o worker, a infraestrutura e os artefatos do processo. (Há um frontend no repositório, mas a interface de vídeo não faz parte do escopo desta fase.)

Ferramenta de IA

O projeto base é construído para o **Claude Code**, que é a ferramenta **recomendada**: toda a fundação de IA (skills, sub-agents, CLAUDE.md, rules e .mcp.json) já vem pronta para ele.

Você **pode** usar outra ferramenta agêntica se preferir (Gemini CLI, OpenAI Codex e similares). Mas atenção: a fundação de IA do repositório é específica do Claude Code. Se optar por outra ferramenta, **é responsabilidade sua portar essa fundação para a convenção dela** antes de começar:

- O CLAUDE.md → o arquivo de instruções equivalente (por exemplo, GEMINI.md no Gemini CLI, AGENTS.md no Codex).
- As skills e sub-agents do workflow → o mecanismo equivalente da ferramenta (comandos, extensões, custom modes) ou, na falta de equivalente, conduza o mesmo workflow manualmente.
- O .mcp.json e os servidores MCP (Postgres, context7) → a configuração de MCP da ferramenta escolhida.

Consulte sempre a **documentação oficial da ferramenta** para os nomes corretos de arquivos, pastas e comandos. Independentemente da ferramenta, o **workflow a seguir é o mesmo** e os **artefatos entregues são os mesmos** (descritos abaixo): só a máquina muda. A escolha da ferramenta não altera os Critérios de Aceite.

Sobre o uso de IA

IA é a ferramenta de produção principal e obrigatória. Seu papel é de **maestro do processo**: conduzir o workflow na ordem certa, revisar criticamente cada saída, refinar prompts quando o resultado vier raso, consultar a documentação das libs antes de implementar e manter os artefatos coerentes entre si.

A presença da IA precisa ser observável no repositório: decisões técnicas, artefatos de planejamento, plano da fase e progresso, tudo gerado pelo fluxo.

Objetivo

Entregar, em um fork público do repositório base, dando continuidade ao projeto mba-ia-greenfield-project:

- Decisões técnicas** da fase (fila, estratégia de upload, streaming, processamento etc.), em docs/decisions/
- Artefatos de planejamento** da fase, na pasta docs/phases/phase-03-videos/ (context.md, validation.md, o plano phase-03-videos.md, o progress.md e o library-refs.md quando houver libs novas a fixar)
- Módulo de vídeos** implementado no backend, com a infraestrutura nova (storage, fila e worker) subindo via Docker
- A Fase 03 funcional**: upload de até 10GB, processamento automático, thumbnail, URL única, streaming e download
- CLAUDE.md (ou o arquivo equivalente da sua ferramenta) atualizado com a seção de vídeos

Toda informação registrada nos artefatos deve ser rastreável ao plano ou ao código. Não invente requisitos, decisões ou comportamentos sem origem identificável.

Contexto

O que já existe no projeto

- Backend **NestJS 11** + **TypeORM** + **PostgreSQL 17** em nestjs-project/, com as Fases 01 e 02 fechadas: módulos auth/, users/, channels/, mail/, common/, config/, database/ e swagger/ (OpenAPI).
- Cada usuário tem um **canal** (relação 1:1), criado no cadastro. Os vídeos da Fase 03 pertencem a um canal.
- Guard JWT global, filtro de exceções de domínio, ValidationPipe global, rate limiting, migrations versionadas e seeds.
- Infra atual no nestjs-project/compose.yaml: **apenas** API, PostgreSQL e Mailpit.
- Um frontend Next.js em next-frontent/ (Fases 01–02), fora do escopo desta fase.

O que **não** existe e você vai construir: o módulo de vídeo, a tabela de vídeos, o **serviço de object storage**, a **fila de processamento** e o **worker de vídeo (FFmpeg)**. A arquitetura-alvo (em docs/diagrams/software-arch.mermaid e no CLAUDE.md) já prevê esses três componentes como parte da Fase 03.

O workflow do projeto

O Luiz desenvolve por um **workflow de planejamento em pipeline**, e você deve segui-lo. Cada estágio é uma skill do projeto e gera um artefato:

Pesquisar opções e decidir o caminho técnico — skill research — artefato docs/decisions/technical-decisions-phase-03-videos.md

Consolidar o contexto da fase — skill plan-context — artefato docs/phases/phase-03-videos/context.md

Validar (inconsistências, decisões faltando, gaps) — skill plan-validate — artefato validation.md (veredito clean/dirty)

Resolver pendências e fixar libs — skill plan-resolve — atualiza decisões/contexto + library-refs.md

Gerar o plano executável — skill plan-build — artefato phase-03-videos.md (Sls + Technical Specs + Dependency Map + Deliverables)

Gerar specs de teste (opcional) — skill plan-test-specs — artefato specs de teste

Implementar passo a passo — skill implement — artefato código + progress.md

Pontos do formato que você precisa respeitar:

- A fase é uma **pasta** (docs/phases/phase-03-videos/) com context.md, validation.md, o plano phase-03-videos.md e o progress.md — mais o library-refs.md quando a fase fixa bibliotecas novas (o caso aqui, por causa de storage/fila/FFmpeg). Use a pasta docs/phases/phase-02-auth/ como referência de formato.
 - O plano é organizado em **Step Implementations** (SI-03.1, SI-03.2, ...) com as **Technical Specifications** (Data Model, API Contracts, Authorization Matrix, Error Catalog e, por causa da fila, **Events/Messages**), o Dependency Map e os Deliverables.
 - O validation.md precisa terminar com **status clean** antes de implementar.
 - Os **sub-agents** de leitura (.claude/agents/) são usados pelas skills por baixo dos panos: você não os invoca diretamente.
- Para quem usa outra ferramenta: produza os **mesmos artefatos** (decisões, contexto, validação, plano com Sls e Technical Specs, progresso), seguindo o mesmo encadeamento. O formato da pasta da fase é o contrato; a skill que o gera é detalhe da ferramenta.

Regras e Definition of Done

O CLAUDE.md define as regras do projeto, que valem para esta fase:

- Definition of Done**: a fase só está pronta quando a suíte de testes relevante passa (unit + integração + e2e), a suíte completa passa, npx tsc --noEmit sai com código 0 e npm run lint passa.
- Docker**: tudo roda em containers; use sempre o **nome do serviço** do Compose como host (ex.: db), nunca localhost.
- Documentação de libs**: antes de implementar com qualquer biblioteca, consulte a doc oficial via **context7** (MCP) e siga a versão instalada.
- Git Flow**: branches de feature/* saem da dev e voltam para a dev; nunca committe direto na main. Commits curtos e descritivos.
- Testes**: sufixos *.spec.ts (unit), *.integration-spec.ts (integração com banco/serviços reais), *.e2e-spec.ts (e2e via supertest).

Escopo da Fase 03 — Upload e Processamento de Vídeos

Você vai entregar as capacidades abaixo (definição completa em docs/project-plan.md, Fase 03):

- Object storage** para os arquivos de vídeo e thumbnails.
- Fila de processamento** em segundo plano e um **worker** que a consome.
- Upload de vídeos de até 10GB sem travar o sistema** (sem segurar a API durante o envio).
- Pré-cadastro automático do vídeo como rascunho** ao iniciar o upload.
- Processamento automático após o upload**: extração de duração e metadados.
- Geração automática de thumbnail** a partir de um frame do vídeo.
- URL única por vídeo**, sem conflito com outros.
- Reprodução via streaming** (sem exigir o download completo).
- Download do vídeo** pelo usuário.

Entregáveis (do plano original): upload de até 10GB funcional, processamento automático do vídeo, streaming funcionando e URLs únicas geradas.

Persistência: uma entidade/tabela de **vídeos** ligada ao canal, com pelo menos identificação, dono (canal), título, **status** (ex.: rascunho → processando → pronto/erro), chaves de storage do arquivo e do thumbnail, duração e metadados, e o identificador da URL única. O modelo exato é definido no plano (Data Model).

Decisões que você precisa tomar e justificar na etapa de research:

- A **tecnologia de fila** — o plano do projeto a deixa explicitamente em aberto ("TBD"). É a principal decisao de stack da fase.
 - A **estratégia de upload de 10GB sem travar** (ex.: upload direto ao storage via URL pré-assinada / multipart, em vez de passar o arquivo pela API).
 - Como o **worker** roda (processo/container separado) e como ele extrai metadados e gera o thumbnail (FFmpeg/ffprobe).
 - A estratégia de **URL única** e a de **streaming** (ex.: requisições com range / 206 Partial Content).
 - O ciclo de **status** do vídeo e o que acontece em caso de falha no processamento.
- Sobre o object storage**: ele não é uma escolha em aberto. O projeto já aponta para **S3 (compatível)** — na prática você roda **MinIO** localmente em Docker (mesma API do S3) e trocaria por S3 em produção. O que você decide aqui é como usá-lo (organização de buckets/chaves, upload pré-assinado), não *qual* storage. A decisão de stack genuinamente aberta é a **fila**.
- Essas decisões são o coração da etapa de research. Pesquise as opções, registre os trade-offs e a escolha no documento de decisões, e só então parta para o planejamento.

Requisitos

1. Decisões técnicas (research)

Rode a etapa de **research** sobre as decisões em aberto acima. Salve em docs/decisions/technical-decisions-phase-03-videos.md, no formato dos documentos de decisão existentes (opções, trade-offs e recomendação por decisão). É esse documento que alimenta o planejamento.

2. Planejamento (pipeline)

Conduza a pipeline de planejamento até o plano final, gerando os artefatos na pasta docs/phases/phase-03-videos/:

- plan-context → context.md
 - plan-validate → validation.md (precisa fechar em **clean**)
 - plan-resolve → resolve as pendências apontadas e gera library-refs.md (libs confirmadas via context7)
 - plan-build → phase-03-videos.md, com Step Implementations (SI-03.x), Technical Specifications (incluindo **Events/Messages** para a fila), Dependency Map e Deliverables
- Revise criticamente cada saída. O validation.md aponta decisões faltando e gaps de dependência: itere **validate** → **resolve** até ficar **clean**. Plano frouxo gera implementação frouxa.

3. Implementação (implement)

Implemente a fase conduzido pela skill **implement**. Sl a SI, rodando os testes a cada passo e só avançando com a suíte do SI verde. Isso inclui:

- O **módulo de vídeos** no backend, seguindo as **convenções e rules do projeto** (separação de camadas, repository pattern, uso de fila/eventos, transações). Use o módulo auth/ como referência de forma; a estrutura concreta de arquivos é decisão do seu plano, não do enunciado.
- A **infraestrutura nova** no **compose.yaml**: serviços de **object storage**, **fila** e **worker**, subindo junto com a stack do backend.
- A **migration** criando a tabela de vídeos.
- Os **testes** nos níveis adequados (unit, integração com banco/serviços reais, e2e), conforme as skills de teste do projeto. Não mocke o que dá para testar de verdade com a infra do Compose.
- O progress.md da fase atualizado (status + testes por SI), como na Fase 02.

Ao final, a **Definition of Done** do CLAUDE.md precisa passar inteira.

4. Atualização da documentação de IA

Atualize o CLAUDE.md (ou o arquivo equivalente da sua ferramenta) refletindo o estado real do código após a fase: o módulo de vídeos, os endpoints, a fila/worker e o storage. Documentação que cite arquivos ou comportamentos inexistentes reprova.

Critérios de Aceite

Todos obrigatórios. Esta é a lista única de avaliação.

Decisões e planejamento

- technical-decisions-phase-03-videos.md com as decisões em aberto resolvidas e justificadas (fila, estratégia de upload, streaming, processamento/thumbnail, ciclo de status)
- Pasta docs/phases/phase-03-videos/ com context.md, validation.md (status clean), o plano phase-03-videos.md, o progress.md e o library-refs.md (se houver libs novas a fixar — esperado nesta fase)
- O plano segue o formato do projeto: Sls SI-03.x, Technical Specifications (Data Model, API Contracts, Authorization Matrix, Error Catalog, Events/Messages), Dependency Map e Deliverables

Implementação — feature

- Upload de vídeo de **até 10GB sem travar** a API, com pré-cadastro do vídeo como rascunho ao iniciar
- Processamento automático após o upload: extração de duração/metadados e geração de thumbnail
- URL única** por vídeo, sem conflito
- Streaming** funcionando (sem exigir download completo) e **download** do vídeo disponível
- Ciclo de status do vídeo (rascunho → processando → pronto/erro) refletido no banco

Implementação — infraestrutura e qualidade

- Object storage, fila e worker **subindo via docker** **compose** junto com o backend
- Migration cria a tabela de vídeos; entidade ligada ao canal
- Testes nos níveis adequados, verdes (npm test e npm run test:e2e)
- Definition of Done** completa: suíte verde + npx tsc --noEmit (código 0) + npm run lint
- Git Flow respeitado (trabalha em feature/* a partir de dev, sem commit direto na main)

Documentação e ferramenta

- CLAUDE.md (ou equivalente) atualizado com a seção de vídeos, coerente com o código
- Se usou outra ferramenta que não o Claude Code: a fundação de IA foi portada para a convenção dela e os artefatos da pasta da fase foram entregues no mesmo formato

Reprova automática

- Pular o workflow: implementar sem as etapas de research, planejamento e implementação (e seus artefatos)
- Plano sem Sls ou sem as Technical Specifications, ou validation.md que não fecha em **clean**
- Passar o arquivo de 10GB pela API de forma que trave o sistema (sem estratégia de upload assíncrono/direto)
- Não ter fila, worker e storage reais subindo no Compose
- tsc com erro, lint quebrado ou suíte vermelha
- Commit direto na main
- CLAUDE.md/equivalente inconsistente com o código
- Usar outra ferramenta sem portar a fundação para a convenção dela

Estrutura do entregável

Tudo dentro do fork do mba-ia-greenfield-project. Abaixo, apenas o que é novo/alterado (os seus arquivos do módulo são infortativos — a estrutura final é decisão do seu plano):

```
mba-ia-greenfield-project/
├── docs/
│   ├── decisions/
│   │   └── technical-decisions-phase-03-videos.md  ← research
│   └── phases/
│       └── phase-03-videos/
│           ├── context.md
│           ├── validation.md
│           ├── library-refs.md
│           ├── phase-03-videos.md
│           └── progress.md
├── nestjs-project/
│   ├── CLAUDE.md (ou equivalente)
│   ├── compose.yaml
│   └── src/
│       ├── videos/
│       │   ├── ...
│       │   └── database/migrations/CreateVideos.ts
│       └── (worker de vídeo - local conforme o seu plano)
└── CLAUDE.md (ou equivalente)  ← atualizado
```

Repositório base

<https://github.com/devfullcycle/mba-ia-greenfield-project>

O fork é a sua estrutura de trabalho: você não cria um repositório novo, apenas adiciona/edita arquivos dentro dele. O repositório já traz as Fases 01 e 02 (backend e frontend), o workflow completo em .claude/ (skills, sub-agents e rules), o CLAUDE.md, o docs/project-plan.md e o compose.yaml do backend com Postgres e Mailpit.

Ordem de execução sugerida

- Setup**. Faça o fork, suba o backend (cd nestjs-project && docker compose up -d, instale as dependências, rode as migrations) e confirme a suíte atual verde. Se for usar outra ferramenta, porte a fundação de IA antes de começar.
- Research**. Pesquise e feche as decisões em aberto (fila, estratégia de upload, streaming, processamento). O object storage já é dado (S3/MinIO).
- Planejamento**. Rode a pipeline (context → validate → resolve → build) até o validation.md fechar em **clean** e o plano ficar completo. Revise criticamente.
- Implementação**. Conduza a implementação SI a SI: módulo, infra no Compose, migration, testes e progress.md.
- Fechamento**. Garanta a Definition of Done (testes + tsc + lint), atualize o CLAUDE.md e revise os Critérios de Aceite item a item antes do push.

Dicas finais

- A **Fase 03 é grande**; o plano é o que **segura**. Quanto melhores as decisões e o plano (Sls bem batidos, contratos e eventos definidos), mais limpa a implementação. Gaste tempo no planejamento.
- Upload de 10GB é decisão de arquitetura, não de força bruta**. Pesquise a estratégia certa antes de codar; passar o arquivo inteiro pela API é o caminho errado.
- Infra real, testada**. Fila, worker e storage precisam subir no Compose e ser exercitados pelos testes: não simule o que dá para rodar de verdade.
- Continuidade, não retrabalho**. Reuse os padrões do projeto (guard, filtro de exceções, repository, migrations, rules). Você está somando uma fase, não reescrevendo o que já existe.
- Ferramenta é escolha sua, workflow não**. Use o Claude Code ou porte a fundação para a sua ferramenta — mas o encadeamento research → planejamento → implementação e os artefatos da fase são os mesmos.

Área de entrega

Insira o link do repositório

Enviar